

PROGRAMMING ASSIGNMENT 12

Machine Learning, Dimensionality Reduction, Clustering & Regression

Superconductivity Dataset | Python 3 | Scikit-Learn

Assignment Details	
Course	Machine Learning and Data Analysis
Assignment No.	12
Dataset	Superconductor Dataset (Hamidieh, 2018) — 10,000 samples, 81 features
Target	critical_temp — Critical Temperature in Kelvin (regression task)
Language	Python 3.x
Libraries	pandas, numpy, matplotlib, seaborn, scikit-learn, scipy, statsmodels
Submission	Jupyter Notebook (.ipynb) with all outputs and markdown analysis
Structure	Part A: Visualization, Feature Engineering & Clustering Part B: Regression

Dataset Overview

The Superconductivity Dataset was introduced by Kam Hamidieh (2018) and is hosted on the UCI Machine Learning Repository (ID: 464). It contains records for 10,000 superconducting materials sourced from Japan's National Institute for Materials Science (SuperCon database).

Feature structure — 81 features follow a 9×9 pattern:

- 9 elemental properties: atomic mass, atomic radius, valence, electron affinity, ionization energy (fie), thermal conductivity, density, fusion heat, number of elements
- 9 statistical descriptors per property: mean, wtd_mean, gmean, wtd_gmean, entropy, wtd_entropy, range, wtd_range, std/wtd_std

Target variable — critical_temp: the temperature (Kelvin) below which a material exhibits zero electrical resistance. This is a continuous numeric value making the primary task a regression problem. The distribution is right-skewed, motivating log-transformation before modelling.

Assignment Objective

This assignment develops competence in the full unsupervised and supervised ML pipeline:

- Apply t-SNE and PCA for exploratory visualisation of high-dimensional chemical data
- Use Variance Inflation Factor (VIF) to identify and remove multicollinear features
- Apply K-Means and Hierarchical Agglomerative Clustering on the reduced feature set
- Build, compare, and evaluate multiple regression models including ensemble methods
- Implement a complete Scikit-Learn Pipeline integrating PCA and Random Forest
- Interpret and explain every result, visualisation, and model decision

Instructions for Students

- **Load the dataset from file:** `pd.read_csv('Superconductor_Dataset_10000.csv')`
- Set `random_state=42` wherever randomness is involved (t-SNE, K-Means, train/test split, etc.)
- Save every figure with `plt.savefig('filename.png', dpi=150, bbox_inches='tight')` before `plt.show()`
- Include a written interpretation after every major result, plot, or metric table
- All axes must have proper titles, x-labels, y-labels, and legends where applicable
- Submit a single Jupyter Notebook (.ipynb) with all cells executed and outputs visible
- Do NOT hardcode answers — all results must be produced from running code
- Do NOT share code with other students — all submissions will be checked for plagiarism
- **For Coding question :** provide notebook code in 'submission_A12_<Name>.ipynb'
- **For Analysis in each question :** provide .pdf file with analysis done in each coding task. 'submission_A12_<Name>.pdf'

PART A Visualization, Feature Engineering & Clustering

Section 1: Dataset Loading and EDA

Q1. Dataset Loading and Initial Diagnostics

Load the dataset and perform initial diagnostics.

1. Load `Superconductor_Dataset_10000.csv` into a `DataFrame`. Display the first 5 rows and print the shape (rows, columns).
2. Report: total missing values, total duplicate rows, number of numerical features, and number of categorical features.
3. Separate the features matrix `X` (all columns except `critical_temp`) and the target vector `y` (`critical_temp`). Print shapes of both.

Analysis Questions (write your answers in markdown cells):

- Is this a regression or classification problem? Justify your answer using the target variable type.
- What is the 9×9 structure of features? Why does this structure create multicollinearity?
- Are there any data quality issues that must be addressed before modelling?
- What is the range of the target variable `critical_temp`?

Q2. Exploratory Data Analysis (EDA)

Perform comprehensive EDA using the following tasks:

4. Print the statistical summary of all the features present in dataset.
5. Target variable analysis: Generate a 3-panel figure — (a) histogram with mean and median lines, (b) **KDE plot**, (c) boxplot. Save as `01_target_distribution.png`.
6. Feature distributions: Plot histograms for 12 representative features (at least one per elemental property group). Title each subplot with the feature name and its skewness value [use `skew` function of `pandas`]. Save as `02_feature_distributions.png`.
7. Correlation heatmap: Compute the Pearson correlation matrix for all 81 features. Plot a lower-triangle heatmap. Report the number of feature pairs with $|\text{correlation}| \geq 0.90$. Save as `03_correlation_heatmap.png`.

8. Feature-target correlation: Plot a bar chart of the top 20 features most correlated with `critical_temp` (by |Pearson r|). Save as `05_feature_target_corr.png`.

For each visualisation, write an interpretation explaining what the plot reveals about the data.

Required analysis questions:

- Is the target distribution symmetric or right-skewed? What does this imply for regression?
- Why do the correlation heatmap show block patterns?
- Should outliers be removed? Justify your answer in context of the physical data.

Section 2: Preprocessing and VIF Feature Selection

Q3. Data Preprocessing Pipeline

Build a Scikit-Learn preprocessing pipeline.

9. Build a Pipeline with ColumnTransformer containing: SimpleImputer(strategy='median') and StandardScaler(). Apply it to X to produce `X_scaled` as a DataFrame.
10. Print the before-and-after statistics: mean and std of X before and after scaling. Confirm scaling is correct.
11. Print the full pipeline object using `print(full_pipeline)`.

Analysis questions:

- Why is StandardScaler essential before running PCA and K-Means? What happens if you skip this step?
- Why is median imputation preferred over mean imputation for this dataset?
- Are any encoding steps needed here? Why or why not?
- What is the advantage of using Pipeline and ColumnTransformer instead of applying transformations manually?

Q4. VIF-Based Feature Selection

Implement and apply recursive VIF elimination.

12. Implement `recursive_vif_elimination(X_df, vif_limit=5, min_features=5)`. The function must: compute VIF for all features using `variance_inflation_factor()`, identify the feature with the highest VIF, drop it if $VIF \geq 5$, and repeat until all VIFs < 5 or only 5 features remain. Return the reduced DataFrame and the final VIF table.
13. Apply the function to `X_scaled`. Print: how many features were removed, how many remain, and list the remaining features.
14. Visualize VIF before vs after: Plot two horizontal bar charts side by side - top 20 VIF values before elimination, and all features after. Mark the $VIF=5$ threshold line on both. Save as `06_vif_comparison.png`.

Analysis questions:

- How many features were removed (count and percentage)?
- Why does this dataset have such extreme multicollinearity?
- What is the difference between removing correlated feature pairs directly vs. using VIF elimination?

Section 3: Dimensionality Reduction

Q5. Principal Component Analysis (PCA)

Perform full PCA analysis on the 81-feature scaled dataset.

15. Fit `PCA(random_state=42)` on `X_scaled`. Compute explained variance ratio (`evr`) and cumulative explained variance (`cumevr`). Find `n90`: the number of PCs needed to capture 90% of total variance.
16. Print a table showing individual and cumulative variance for the first 15 PCs.
17. Use the above table data and generate a side-by-side Scree Plot and Cumulative Variance Plot. On the cumulative plot, mark the 90% threshold (red dashed line) and the `n90` component (orange vertical line). Save as `07_pca_scree.png`. [A scree plot is a line plot used in principal component analysis (PCA) or factor analysis to visualize the eigenvalues (variance explained) of each component, ordered from largest to smallest.]

Analysis questions:

- How many PCs capture 90% of variance? What does this tell you about redundancy?
- Explain what the scree plot shape tells you about the feature structure.
- How does the 9×9 feature structure contribute to PCA's effectiveness on this dataset?

Q6. PCA and t-SNE Visualization

Visualize the dataset in 2D using both PCA and t-SNE.

18. PCA scatter plot: Plot all samples in PC1 vs PC2 space, coloured by `critical_temp` using `cmap='plasma'`. Label axes with variance percentages. Add a colorbar. Save as `09_pca_2d.png`.
19. t-SNE embedding: Pre-reduce to `n90` PCA components first. Then apply `TSNE(n_components=2, perplexity=50, learning_rate=200, max_iter=1000, random_state=42, init='pca')` to the reduced data. Save as part of `10_pca_vs_tsne.png`.
20. Side-by-side comparison: Plot PCA and t-SNE projections side by side, both coloured by `critical_temp`. Save as `10_pca_vs_tsne.png`.

Comparison analysis questions (answer in a markdown table):

- What cluster structures are visible in t-SNE but not in PCA?
- Why do the t-SNE axes have no interpretable meaning?
- Why is it recommended to pre-reduce with PCA before running t-SNE?
- When would you use PCA vs t-SNE in a real project? Give one scenario for each.

Section 4: Clustering Analysis

Q7. K-Means Clustering

Apply K-Means clustering on the VIF-selected dataset (`X_selected`).

Important: Clustering must be performed on the FULL VIF-selected feature space. PCA (`n=2`) is used ONLY for visualization.

21. Prepare visualization spaces: Fit PCA($n_components=2$) on $X_selected$ for cluster plotting. Also compute a t-SNE embedding ($perplexity=30$, $max_iter=1000$, $init='pca'$) on $X_selected$.
22. K-Means sweep: For k in range(2, 9), fit $KMeans(n_clusters=k, random_state=42, n_init=10)$ on $X_selected$. Compute $silhouette_score$ and $davies_bouldin_score$ on $X_selected$. Print a results table. Save metric plots as `11_kmeans_metrics.png`.
23. Optimal k: Identify the best k from Silhouette Score. Apply final K-Means with this k . Report final Silhouette and Davies-Bouldin scores.
24. Cluster visualization: Plot clusters in PCA space (left) and t-SNE space (right). Color clusters by label. Plot cluster centroids (projected to PCA space) as black X markers. Save as `12_kmeans_clusters.png`.

Analysis questions:

- What is the optimal k by Silhouette? By Davies-Bouldin? Do they agree?
- What physical material families might each cluster represent?
- Is K-Means well-suited to this dataset's cluster shapes? Justify your answer.

Q8. Hierarchical Clustering and Dendrograms

Apply Agglomerative Hierarchical Clustering with 'ward' linkage methods.

25. Linkage evaluation: For Ward linkage, and for k in range(2, 7), compute $silhouette_score$ and $davies_bouldin_score$ on $X_selected$. Print a structured results table showing Method | k | Silhouette | Davies-Bouldin.
26. Dendrogram generation: Use a 300-sample random subset of $X_selected$ (random seed 42). Compute scipy linkage matrices for Ward Linkage methods. Plot a 2×2 dendrogram grid. Mark a recommended cut-off line on each. Save as `13_dendrograms.png`.
27. Visualization: Using the ward linkage method (by Silhouette at $k=3$), visualize clusters in PCA space and t-SNE space side by side. Save as `14_hier_clusters.png`.

Analysis questions:

- How do you determine optimal cluster count from a dendrogram?
- Which linkage method produces the most interpretable dendrogram for this dataset?
- Comment on t-SNE and PCA space visualization at $k=3$
- Is K-Means or Hierarchical more suitable for this dataset? Why?

PART B Regression Modelling Inside Pipeline

Section 5: Regression Problem Setup

Q9. Problem Definition and Train-Test Split

Define the regression problem and split the data.

28. In a markdown cell, define the regression problem: state the target variable, the input features, the prediction objective, and why regression is appropriate (not classification).
29. Split X and y : 80% training, 20% test, $random_state=42$. Print shapes and descriptive statistics for y_train and y_test to confirm the split is representative.

Analysis questions:

- Why is an 80/20 split standard? What are the risks of using a larger or smaller test set?
- Why must the split be performed BEFORE any preprocessing that uses training statistics (e.g., StandardScaler)?
- Are the training and test sets representative? How do you verify this?

Section 6: Model Building and Evaluation

Q10. Multiple Regression Model Comparison

Train and evaluate 7 regression models using identical preprocessing pipelines.

30. For each model below, build a Pipeline: SimpleImputer(strategy='median') → FunctionTransformer(np.log1p) → StandardScaler() → Model. Train on X_train, predict on X_test.
 - Linear Regression
 - Ridge Regression (alpha=1.0)
 - Lasso Regression (alpha=0.1, max_iter=5000)
 - Decision Tree Regressor (max_depth=10, random_state=42)
 - Random Forest Regressor (n_estimators=100, random_state=42)
 - Extra Trees Regressor (n_estimators=100, random_state=42)
 - Gradient Boosting Regressor (n_estimators=100, random_state=42)
31. Print a comparison table: Model | R^2 | MAE | MSE | RMSE. Sort by R^2 descending.
32. Plot a side-by-side horizontal bar chart comparing R^2 and RMSE for all 7 models. Highlight the best model in gold. Label each bar with its value. Save as 16_model_comparison.png.

Analysis questions:

- Which model achieves the best R^2 and lowest RMSE?
- Why do tree-based ensemble models outperform linear models on this dataset?
- Is the Decision Tree overfitting? How can you tell from the metrics?
- Explain the bias-variance trade-off for each model type.
- Why does the log1p transformation improve model performance?

Q11. PCA + RandomForest Pipeline

Implement a full ML pipeline integrating PCAAnalyzer as a custom transformer.

33. Implement PCAAnalyzer(BaseEstimator, TransformerMixin) with parameters threshold=0.90 and plot=True. In fit(): fit full PCA for analysis, fit PCA(n_components=threshold) for transformation, store explained_var_ and cumulative_variance_, and plot scree + cumulative variance charts. In transform(): return self.pca_.transform(X).
34. Build pipeline: SimpleImputer → FunctionTransformer(np.log1p) → StandardScaler → PCAAnalyzer(threshold=0.90, plot=True) → RandomForestRegressor(n_estimators=100, random_state=42). Train and evaluate.
35. Compare pipeline performance against the full-feature Random Forest from Q11. Report the RMSE difference and comment on whether PCA is beneficial for this model.

Analysis questions:

- How many PCA components are retained at the 90% threshold inside the pipeline? Does this match n90 from Part A?
- What is the RMSE cost of adding PCA to the pipeline? Is the dimensionality reduction worth it?
- Explain why adding PCA before a tree-based model may hurt (not help) performance.
- What type of model would benefit MOST from PCA preprocessing?

Final Conclusions (Mandatory)

Q12. Final Report and Conclusions

Write a structured conclusion in a markdown cell covering both parts.

Part A — Feature Engineering and Clustering:

36. Summarise the EDA findings. What were the key data characteristics?
37. How effective was VIF-based feature selection? How many features were removed and why?
38. Compare PCA vs t-SNE: which was more informative, and why?
39. Compare K-Means vs Hierarchical Clustering: which performed better, and which linkage was best?
40. What natural material families were identified by clustering? Relate to known superconductor types.

Part B — Regression Modelling:

41. Rank all 7 regression models. State the best model with its R^2 and RMSE.
42. Explain why ensemble models outperformed linear models on this specific dataset.
43. What is the impact of including PCA in the Random Forest pipeline?

Overall insight:

- Which single preprocessing step had the greatest positive impact on model performance?
- If you had to deploy one model for predicting T_c in a materials lab, which would you choose and why?

End of Assignment 12 Good Luck